

Especificação do Projeto

Sistema de Reserva de Ingressos

1. Visão Geral do Sistema

O sistema consiste em uma API backend voltada para a gestão e reserva de ingressos ou vagas em eventos. A plataforma orquestra o fluxo completo de bilheteria digital, atendendo a três perfis distintos de usuários:

- **Administradores:** Moderam a plataforma, garantindo a integridade do catálogo e gerenciando as permissões de acesso dos demais usuários.
- **Organizadores:** Criam, gerenciam e monitoram a lotação de seus próprios eventos.
- **Compradores:** Exploram o catálogo de eventos ativos e realizam reservas de vagas.

O Desafio Técnico: Esta aplicação proíbe a utilização de ORMs ou query builders (como TypeORM, Sequelize ou Prisma). Toda a comunicação com o banco de dados SQLite deve ser feita por meio da escrita direta de queries em SQL puro. O objetivo central é consolidar o domínio prático sobre a arquitetura interna do NestJS, aplicando Middlewares, Guards, Pipes e Interceptors para garantir a segurança, validação, formatação e consistência transacional do sistema.

2. Requisitos Funcionais e Regras de Negócio

2.1. Autenticação e Gestão de Usuários

- **RF01:** O sistema deve permitir o cadastro público de novos usuários, atribuindo-lhes automaticamente o cargo base de Comprador (BUYER = 0).
- **RF02:** O sistema deve armazenar e validar as senhas de acesso em formato de texto puro (*raw text*) no banco de dados, sem a aplicação de hash criptográfico.
- **RF03:** O sistema deve emitir um token JWT após a validação de login com sucesso, que deverá ser exigido no cabeçalho (Authorization: Bearer <token>) de todas as requisições a rotas protegidas.
- **RF04:** O sistema deve iniciar automaticamente com um usuário Administrador padrão (administrator / administrator) pré-configurado no banco de dados.
- **RF05:** O sistema deve permitir que apenas contas com o cargo de Administrador (ADMINISTRATOR = 3) alterem o cargo de outros usuários (ex: promovendo um Comprador a Organizador).
- **RF06:** O sistema deve bloquear qualquer tentativa de um usuário alterar o seu próprio cargo, independentemente do seu nível de permissão atual.
- **RF07:** O sistema deve permitir que os usuários atualizem seus próprios dados cadastrais básicos (login e senha).

2.2. Gestão do Catálogo de Eventos

- **RF08:** O sistema deve permitir que um Organizador (ORGANIZER = 2) crie novos eventos informando título, descrição, data/hora de início e capacidade total de vagas.

- **RF09:** O sistema deve permitir que Organizadores editem os dados ou cancelem exclusivamente os eventos que foram criados por eles mesmos.
- **RF10:** O sistema deve permitir que Administradores cancelem qualquer evento no sistema para fins de moderação, mas deve impedi-los de editar os dados internos (título, vagas, etc.) desses eventos.
- **RF11:** O sistema deve disponibilizar uma listagem pública de catálogo contendo apenas os eventos com status ativo.
- **RF12:** O sistema deve manter na listagem pública os eventos cuja capacidade de vagas se esgotou, marcando-os de forma explícita com o status de “Esgotado”.

2.3. Sistema de Reservas e Transações

- **RF13:** O sistema deve permitir estritamente 1 (uma) reserva ativa por usuário em um mesmo evento.
- **RF14:** O sistema deve bloquear tentativas de novas inscrições em eventos cuja data e hora de início já tenham sido ultrapassadas.
- **RF15:** O sistema deve processar a reserva de uma vaga (decremento do estoque) e a inserção do registro da reserva de forma atômica, utilizando uma transação isolada de banco de dados (BEGIN IMMEDIATE), mitigando falhas de concorrência e superlotação.
- **RF16:** O sistema deve permitir que um usuário cancele a sua própria inscrição, devolvendo a vaga de forma imediata ao estoque de assentos disponíveis do evento (garantindo que o estoque não ultrapasse a capacidade total original).
- **RF17:** O sistema deve implementar as operações de cancelamento (seja de eventos ou de reservas) utilizando a abordagem de *Soft Delete*, alterando a coluna de status para 'cancelled' em vez de remover a linha fisicamente da tabela.
- **RF18:** O sistema deve propagar o cancelamento de um evento automaticamente para o histórico de reservas de todos os usuários inscritos, refletindo que o evento em si foi cancelado, independentemente do status da participação individual.

3. Recursos de Infraestrutura Estruturais

O sistema exige a implementação de 8 recursos distribuídos pelas camadas funcionais do NestJS:

Middlewares

1. **TraceIdMiddleware:** Gera um UUID v4 único de 36 caracteres para cada requisição recebida e o injeta no cabeçalho de resposta (X-Trace-Id).
2. **RequestLoggerMiddleware:** Registra no console o método HTTP, a URL e o Trace ID de cada chamada processada.

Guards

3. **JwtAuthGuard:** Intercepta as rotas protegidas e bloqueia o fluxo caso o token JWT esteja ausente, malformado ou expirado.
4. **RolesGuard:** Analisa a role contida no token do usuário e valida o acesso com base nas permissões exigidas pela rota através do metadado `@Roles()`.

Pipes

5. **ValidationPipe:** Configurado globalmente com as propriedades de remoção de propriedades não mapeadas (*whitelist*), barra payloads de entrada malformados antes de atingirem os controladores.
6. **ParseUUIDPipe:** Valida os parâmetros de rotas dinâmicas diretamente na URL, rejeitando formatos que não correspondam a um UUID v4 válido.

Interceptors

7. **TimeoutInterceptor:** Estabelece um limite máximo de execução de 30 segundos por rota, convertendo requisições travadas em exceções do tipo 408 Request Timeout.
8. **ResponseTransformInterceptor:** Captura retornos de sucesso dos serviços e padroniza o JSON enviado ao cliente sob o formato: { success: true, data, traceId, timestamp }.

4. Estrutura de Pastas e Componentes Internos

Abaixo está o mapeamento dos diretórios internos e arquivos do projeto que compõem a arquitetura modular da aplicação, com destaque para a separação dos contratos (DTOs) por domínio:

```
src/
├── app.controller.ts
├── app.module.ts
├── app.service.ts
├── auth/
│   ├── auth.controller.ts
│   ├── auth.module.ts
│   ├── auth.service.ts
│   ├── decorators/
│   │   └── roles.decorator.ts
│   ├── guards/
│   │   ├── jwt-auth.guard.ts
│   │   └── roles.guard.ts
│   └── index.ts
├── common/
│   ├── filters/
│   │   ├── all-exceptions.filter.ts
│   │   └── index.ts
│   ├── interceptors/
│   │   ├── index.ts
│   │   ├── response-transform.interceptor.ts
│   │   └── timeout.interceptor.ts
│   ├── middleware/
│   │   ├── request-logger.middleware.ts
│   │   └── trace-id.middleware.ts
│   └── types/
│       ├── auth-user.interface.ts
│       ├── entity-status.type.ts
│       ├── index.ts
│       ├── role.enum.ts
│       └── rows.ts
├── config/
│   └── env.validation.ts
├── database/
│   └── database.module.ts
```

```
|   |   | database.service.ts
|   |   |
|   |   | dto/
|   |   |   |
|   |   |   | auth/
|   |   |   |   |
|   |   |   |   | index.ts
|   |   |   |   | login.dto.ts
|   |   |   |   | register-user.dto.ts
|   |   |   |   |
|   |   |   |   | common/
|   |   |   |   |   |
|   |   |   |   |   | api-envelope.dto.ts
|   |   |   |   |   | index.ts
|   |   |   |   |   |
|   |   |   |   |   | events/
|   |   |   |   |   |   |
|   |   |   |   |   |   | create-event.dto.ts
|   |   |   |   |   |   | event-view.dto.ts
|   |   |   |   |   |   | index.ts
|   |   |   |   |   |   | update-event.dto.ts
|   |   |   |   |   |   |
|   |   |   |   |   |   | index.ts
|   |   |   |   |   |   |
|   |   |   |   |   |   | reservations/
|   |   |   |   |   |   |   |
|   |   |   |   |   |   |   | index.ts
|   |   |   |   |   |   |   | reservation-history-entry.dto.ts
|   |   |   |   |   |   |   |
|   |   |   |   |   |   |   | users/
|   |   |   |   |   |   |   |   |
|   |   |   |   |   |   |   |   | change-role.dto.ts
|   |   |   |   |   |   |   |   | index.ts
|   |   |   |   |   |   |   |   | update-user.dto.ts
|   |   |   |   |   |   |   |   |
|   |   |   |   |   |   |   |   | validators/
|   |   |   |   |   |   |   |   | is-future-date.validator.ts
|   |   |   |   |   |   |   |   |
|   |   |   |   |   |   |   |   | events/
|   |   |   |   |   |   |   |   |   |
|   |   |   |   |   |   |   |   |   | events.controller.ts
|   |   |   |   |   |   |   |   |   | events.module.ts
|   |   |   |   |   |   |   |   |   | events.service.ts
|   |   |   |   |   |   |   |   |
|   |   |   |   |   |   |   |   | main.ts
|   |   |   |   |   |   |   |   |
|   |   |   |   |   |   |   |   | reservations/
|   |   |   |   |   |   |   |   |   |
|   |   |   |   |   |   |   |   |   | reservations.controller.ts
|   |   |   |   |   |   |   |   |   | reservations.module.ts
|   |   |   |   |   |   |   |   |   | reservations.service.ts
|   |   |   |   |   |   |   |   |   |
|   |   |   |   |   |   |   |   |   | users/
|   |   |   |   |   |   |   |   |   |   |
|   |   |   |   |   |   |   |   |   |   | users.controller.ts
|   |   |   |   |   |   |   |   |   |   | users.module.ts
|   |   |   |   |   |   |   |   |   |   | users.service.ts
```